

AGL AI SYSTEMS

Puks AI + Commo

Project Overview & Knowledge Transfer

Prepared by: Kgathola Puka

Table of Contents

- Table of Contents 2
- 1. Why This Project Exists..... 3
- 2. System 1 - Puks AI (Document Intelligence) 4
 - 2.1 What it knows 4
 - 2.2 What it cannot do..... 4
 - 2.3 The knowledge base files 4
 - 2.4 The 6 JSON knowledge base files (domain schemas) 5
- 3. System 2 - Commo (Live Data Intelligence)..... 6
 - 3.1 What makes Commo different from Puks AI 6
 - 3.2 The data pipeline 6
 - 3.3 The schema layer - Commo's knowledge base 6
 - 3.4 Country rollout plan..... 6
- 4. Architecture - How Both Systems Fit Together 8
 - 4.1 Key architectural decisions and why..... 8
 - 4.2 What was intentionally NOT built..... 9

1. Why This Project Exists

Speed WMS is the warehouse management system used by AGL across multiple African countries. It is a complex system with a large database containing many interrelated tables, a French-language codebase, and limited self-service documentation.

Two recurring main points were identified that this project addresses:

Main point	How this project solves it
Support team spent hours answering repetitive WMS questions - how I create a GRN, how do I know when stock is generated, what a status code means	Puks AI - a chatbot trained on WMS documentation that answers these questions instantly.
Operations and management had no self-service access to live warehouse data, every data question required a developer or a scheduled BI report	Commo - a chatbot that converts plain English questions into SQL, runs them against live Fabric data, and returns answers instantly

2. System 1 - Puks AI (RAG)

Puks AI stands for Predictive Unified Knowledge System. It is a RAG (Retrieval-Augmented Generation) chatbot that answers questions from a static knowledge base of Speed WMS documentation.

2.1 Data Used to Train

- All Speed WMS table schemas - column names, data types, primary keys, foreign key relationships
- Join logic between tables - how REE_DAT connects to REL_DAT, how MIE_DAT connects to MIL_DAT etc.
- Operational procedures - step-by-step WMS workflows with exact SQL included
- Status codes - what each code value means in each domain (inbound, outbound, loading, inventory)
- Safety rules - which joins are forbidden and why, named column exceptions
- General WMS naming conventions - _DAT, _PAR, _BAS, _SYS suffixes mean

2.2 What it cannot do

- Query live data - it only knows about documentation, not current warehouse state
- Answer questions outside the knowledge base - it will refuse rather than guess
- Make changes to any system - it is read-only by design

2.3 The knowledge base files

The knowledge base is stored in two places:

- DATA/ folder - raw PDFs, JSON schemas, WMS reference JSONs
- Azure Blob Storage (agl-ai-storage, container: puks-data) - processed files used at runtime

The 4 runtime files in Blob Storage:

File	What it contains
faiss.index	FAISS vector index - all WMS documentation as mathematical vectors for semantic search
metadata.pkl	Chunk metadata aligned to the FAISS index - must be from the same embed.py run as faiss.index
unified_chunks.json	All processed text chunks with metadata - used for BM25 keyword search and runtime reference

config.json	Model name, vector dimensions, chunk type counts - read by Function App at startup
--------------------	--

2.4 The 6 JSON knowledge base files (domain schemas)

These are manually curated JSON files that describe the Speed WMS database in structured format. They are the primary source of schema knowledge for Puks AI. Located in: DATA/schemas/

File	What it covers
01_general_rules.json	Speed WMS naming conventions - _DAT/_PAR/_BAS/_SYS suffixes
02_base_data.json	Warehouse mapping, article tables, stock tables and joins
03_inbound.json	Reception tables (REE_DAT, REL_DAT), inbound joins, status codes
04_outbound.json	Order/mission/expedition tables, outbound joins, status codes
05_loading.json	Loading/chargement tables (CHG_DAT, CHL_DAT), safety rules
06_inventory.json	Inventory tables (IVE_DAT, IVL_DAT) and joins

3. System 2 - Commo

Commo is a Text-to-SQL AI chatbot. It converts plain English questions into SQL queries, executes them against live Speed WMS data via Microsoft Fabric, and returns plain English answers.

3.1 What makes Commo different from Puks AI

- Puks AI answers from static documents. Commo answers from live data - stock levels, movements, orders as they exist right now.
- Puks AI never touches a database. Commo executes SQL against Microsoft Fabric.
- Puks AI is for technical WMS questions. Commo is for business data questions.

3.2 The data pipeline

Speed WMS data flows from source to Commo in this sequence:

1. Speed WMS runs on Azure SQL - this is the source of truth, managed by IT
2. Microsoft Fabric Mirroring creates a near real-time copy of Azure SQL inside a Fabric Lakehouse - 1–2minute lag, automatic, no code required
3. The Fabric Lakehouse stores mirrored data as Delta tables - queryable via SQL
4. Commo queries ONLY the Fabric SQL Analytics Endpoint - never Azure SQL directly
5. A SQL validator blocks all non-SELECT statements before execution - Commo cannot modify data

3.3 The schema layer - Commo's knowledge base

Unlike Puks AI which searches documents, Commo uses a schema layer to understand what tables and columns exist. This schema layer must be maintained manually when the database changes.

- `schema_registry.json` - table and column descriptions in plain English
- `glossary.json` - maps business terms to database values (e.g. 'Ivory Coast' = `COUNTRY_CODE 'CI'`)
- `examples.json` - minimum 30 real question-to-SQL pairs covering all query types

These files are embedded with FAISS (same as Puks AI) so Commo can retrieve the most relevant schema context before generating SQL.

3.4 Country rollout plan

- Phase 1 (current): Ivory Coast - Lakehouse: SpeedWMS_CIV
- Phase 2: Ghana - new Lakehouse SpeedWMS_GHA, extend Fabric Mirroring, update glossary.json
- Phase 3: Senegal and other countries - same pattern
- No architecture changes needed for new countries - just new Lakehouses and updated schema files

4. Architecture

Both systems share the same Microsoft infrastructure. Everything runs inside AGL's Azure tenant and Microsoft 365 environment.

Layer	Puks AI	Commo
UI	Power Apps - canvas app, scrPuks screen	Power Apps - canvas app, scrCommo screen
Bridge	Power Automate - PuksAI Chat Flow + Clear Session Flow	Power Automate - Commo Chat Flow + Clear Session Flow
Backend	Azure Function App: puks-ai-func (Python 3.11)	Azure Function App: commo-ai-func (Python 3.11)
AI Model	Azure OpenAI GPT-4o - shared resource: agl-azure-openai	Azure OpenAI GPT-4o - same shared resource
Data	Azure Blob Storage: agl-ai-storage / puks-data container	Microsoft Fabric Lakehouse via SQL Analytics Endpoint
Auth	Azure Function Key in HTTP header (x-functions-key)	Service Principal: SpeedWMS-Chatbot (Entra ID app registration)
Resource Group	rg-agl-ai (both systems in one group)	rg-agl-ai (shared)

4.1 Key architectural decisions

- Azure Function App (not VM or App Service): Serverless - zero cost when idle, no server to manage, scales automatically. Consumption plan only charges per execution.
- Azure OpenAI: Data stays inside AGL's Azure tenant. No external API calls. Enterprise SLA. No new vendor approval needed.
- Power Apps (not Streamlit): Streamlit cannot be hosted inside AGL's network.
- Fabric: Native Microsoft feature, near real-time, no code, automatic. No pipeline to maintain.
- RAG with FAISS + BM25 + reranker (not simple search): Three-layer retrieval produces significantly more accurate results than keyword search alone. Critical for WMS queries where exact table/column names matter.
- SQL Validator before execution (Commo): Unconditional - no exceptions. Blocks INSERT, UPDATE, DELETE, DROP at all times. This is a core safety feature, not optional.

4.2 Future Upgrades

- No authentication layer beyond Azure Function Key and Power Apps Azure AD - adding row-level security per country is on the roadmap (Phase 4) but not implemented
- No conversation history persistence - memory is in-memory only, cleared on Function App restart
- No web scraping or auto-update of the knowledge base - documentation updates require manual re-running of the pipeline